LAB MANUAL
PART B

Roll No. C-60	**Name: Avdhut Madhavi**
Class: C	**Batch: C2**
Date of Experiment:	**Date of Submission:**
Grade:	

B.1: Implement and trace the following programs. Write the code, compile/run it, and record the output after each task.

1. **Write a C program to implement Selection Sort on an array of integers entered by the user, and display the array after every pass.**

```
#include <stdio.h>
#include <conio.h>
int main() {
   int a[100], n, i, j, min, temp;

   printf("Enter number of elements: ");
   scanf("%d", &n);

   printf("Enter array elements: ");
   for (i = 0; i < n; i++) {
      scanf("%d", &a[i]);
   }

   for (i = 0; i < n - 1; i++) {
      min = i;

      for (j = i + 1; j < n; j++) {
         if (a[j] < a[min]) {
            min = j;
         }
      }

      temp = a[i];
      a[i] = a[min];
      a[min] = temp;

      printf("Pass %d: ", i + 1);
      for (j = 0; j < n; j++) {
         printf("%d ", a[j]);
      }
      printf("\n");
   }

   printf("Sorted array: ");
   for (i = 0; i < n; i++) {
```

```
        printf("%d ", a[i]);
    }
    getch();
    return 0;
}
```

```
C:\TURBOC3\BIN>TC
Enter number of elements: 5
Enter array elements: 34
45
67
89
12
Pass 1: 12 45 67 89 34
Pass 2: 12 34 67 89 45
Pass 3: 12 34 45 89 67
Pass 4: 12 34 45 67 89
Sorted array: 12 34 45 67 89 _
```

2. **Modify the Selection Sort program to count and display the total number of comparisons and swaps performed.**

```
#include <stdio.h>
#include <conio.h>
int main() {
    int a[100], n, i, j, min, temp;
    int comparisons = 0, swaps = 0;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter array elements: ");
    for (i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }

    for (i = 0; i < n - 1; i++) {
        min = i;

        for (j = i + 1; j < n; j++) {
            comparisons++;
```

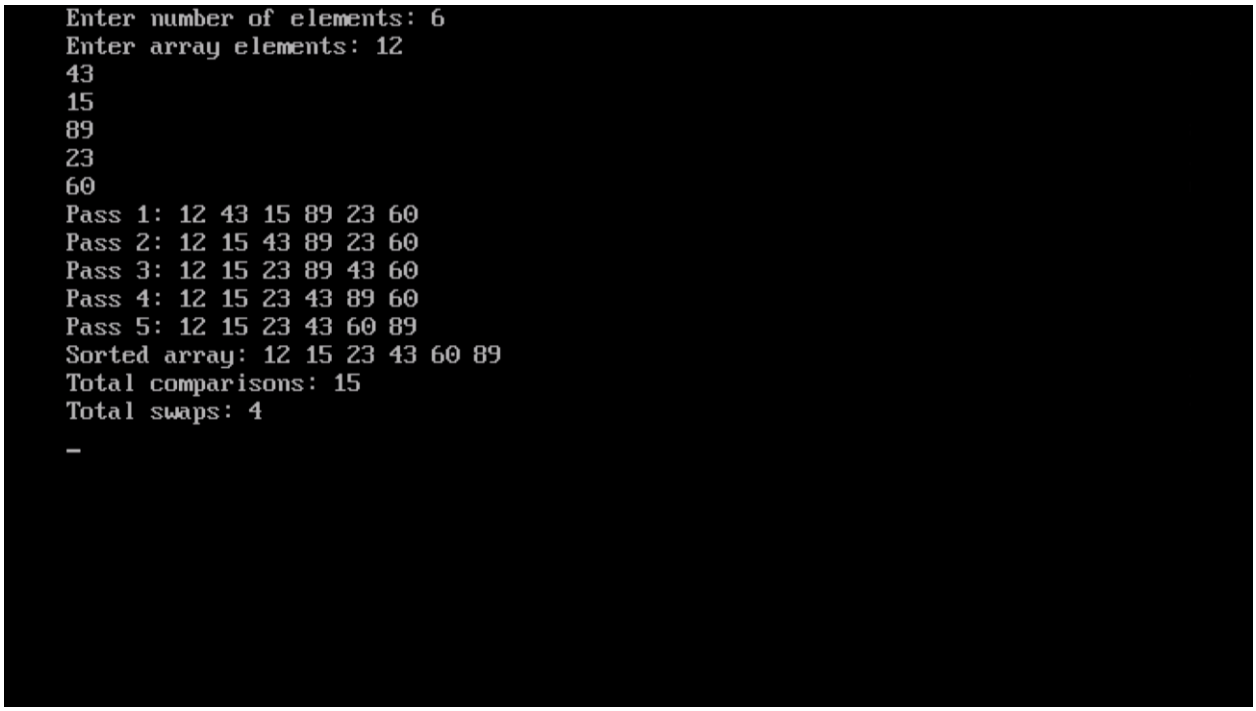
```
            if (a[j] < a[min]) {
                min = j;
            }
        }

        if (min != i) {
            temp = a[i];
            a[i] = a[min];
            a[min] = temp;
            swaps++;
        }

        printf("Pass %d: ", i + 1);
        for (j = 0; j < n; j++) {
            printf("%d ", a[j]);
        }
        printf("\n");
    }

    printf("Sorted array: ");
    for (i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }

    printf("\nTotal comparisons: %d", comparisons);
    printf("\nTotal swaps: %d\n", swaps);
    getch();
    return 0;
}
```

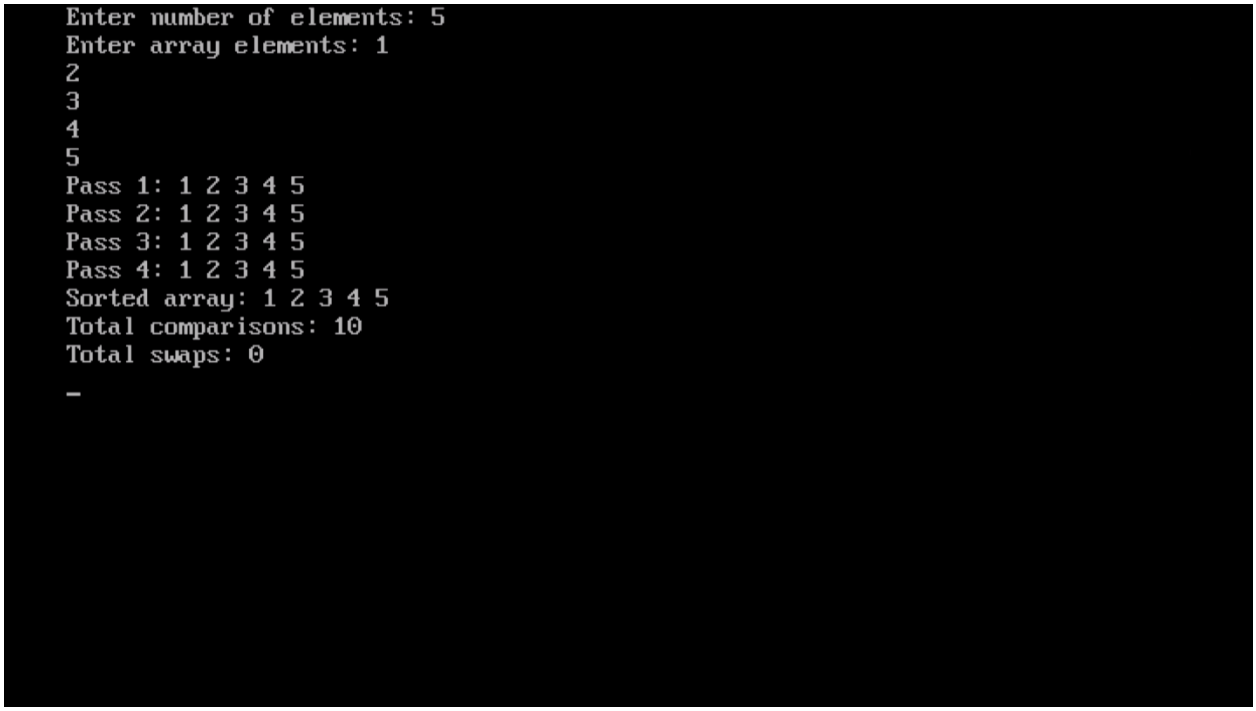


3. **Run both programs on the same already-sorted array (e.g., 1, 2, 3, 4, 5) and compare the number of operations each algorithm performs.**

   Input array:
   1 2 3 4 5
   Selection Sort still searches the entire unsorted portion on every pass, even when the array is already sorted.
   The number of comparisons is:

$$\frac{n(n-1)}{2} = \frac{5(5-1)}{2} = 10$$

   Since the smallest element is already at the beginning of each pass, no actual swaps are needed.



Conclusion: Both programs perform the same sorting operations. Program 2 additionally counts and displays the operations, making it useful for analyzing algorithm performance.

4. **Trace Selection Sort and Insertion Sort by hand on the array [8, 3, 5, 1, 9, 2], showing the array state after each pass.**

Input array: [8, 3, 5, 1, 9, 2]
Pass 1: Find the smallest element, 1, and swap it with 8.
Array: [1, 3, 5, 8, 9, 2]
Pass 2: Find the smallest element in the remaining unsorted part, 2, and swap it with 3.
Array: [1, 2, 5, 8, 9, 3]
Pass 3: Find the smallest element, 3, and swap it with 5.
Array: [1, 2, 3, 8, 9, 5]
Pass 4: Find the smallest element, 5, and swap it with 8.
Array: [1, 2, 3, 5, 9, 8]
Pass 5: Find the smallest element, 8, and swap it with 9.
Array: [1, 2, 3, 5, 8, 9]
Final sorted array: [1, 2, 3, 5, 8, 9]

5. Hand Trace of Insertion Sort
Input array: [8, 3, 5, 1, 9, 2]
Pass 1: Take 3 as the key and insert it before 8.
Array: [3, 8, 5, 1, 9, 2]
Pass 2: Take 5 as the key and insert it between 3 and 8.
Array: [3, 5, 8, 1, 9, 2]
Pass 3: Take 1 as the key and insert it at the beginning.
Array: [1, 3, 5, 8, 9, 2]
Pass 4: Take 9 as the key. It is already in the correct position.
Array: [1, 3, 5, 8, 9, 2]
Pass 5: Take 2 as the key and insert it after 1.

Array: [1, 2, 3, 5, 8, 9]
Final sorted array: [1, 2, 3, 5, 8, 9]

6. Time and Space Complexity

Algorithm	Best Case	Average Case	Worst Case	Space
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(1)
Insertion Sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)

Conclusion: Selection Sort always performs $O(n^2)$ comparisons. Insertion Sort is more efficient for already-sorted or nearly sorted arrays because its best-case time complexity is O(n).

B.2: Write an observation according to the program execution.

- Selection Sort sorts the array in ascending order by repeatedly finding the minimum element.
- Insertion Sort sorts the array by inserting each element into its correct position.
- Selection Sort performs $O(n^2)$ comparisons, while Insertion Sort has a best-case time complexity of O(n).
- Both algorithms successfully sort the given array.